

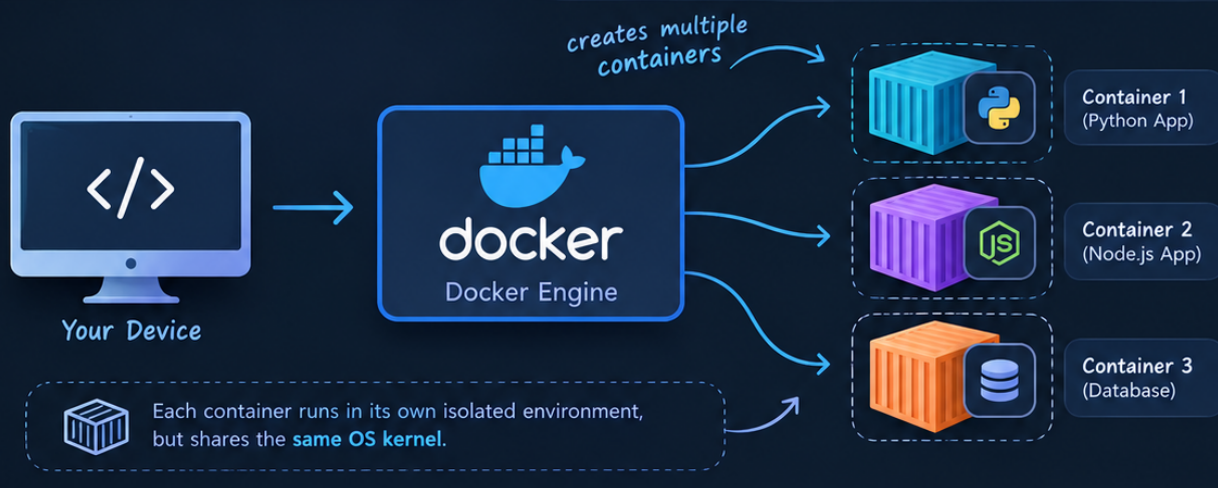


Docker is a platform that lets you build, package and run applications in **isolated containers**. It makes your development, testing and deployment process simple, fast and consistent.

What is a Container?

A container is a lightweight, standalone package that includes your application and all its dependencies (code, runtime, libraries, etc.).

- ✓ Isolated from other apps
- ✓ Works the same everywhere
- ✓ Small and fast



Why Use Docker?

Fast
Quick setup and deployment

Consistent
Same environment everywhere

Scalable
Run more containers as needed

Flexible
Works on any system (Windows, Mac, Linux)

Efficient
Uses less resources than VMs

— One Docker. Many Possibilities. —

Docker Tutorial for Beginners — Complete One-Shot

Structured study notes with original diagrams, commands, architecture, Docker Compose, Dockerfile, networking and volumes.

Source basis: the linked Apna College video and its accessible transcript summary. The notes follow the video topics and practical flow; they are a study summary, not a transcript.

1. Learning Roadmap

The lesson moves from the problem Docker solves to practical container management and finally to multi-container applications, custom images and persistent data.

Stage	Main topic
1	Why Docker is needed / environment mismatch
2	Containers and images
3	Docker Desktop setup
4	Basic commands and Docker Hub
5	Tags, versions and image layers
6	Ports, environment variables and names
7	Networking with MongoDB + Mongo Express
8	Docker Compose
9	Dockerfile and custom image
10	Docker Hub publishing
11	Volumes and data persistence

Big picture

The Problem Docker Solves

Developer A
Node 16 + Redis 6

Developer B
Node 20 + Redis 7

Dependency / environment mismatch

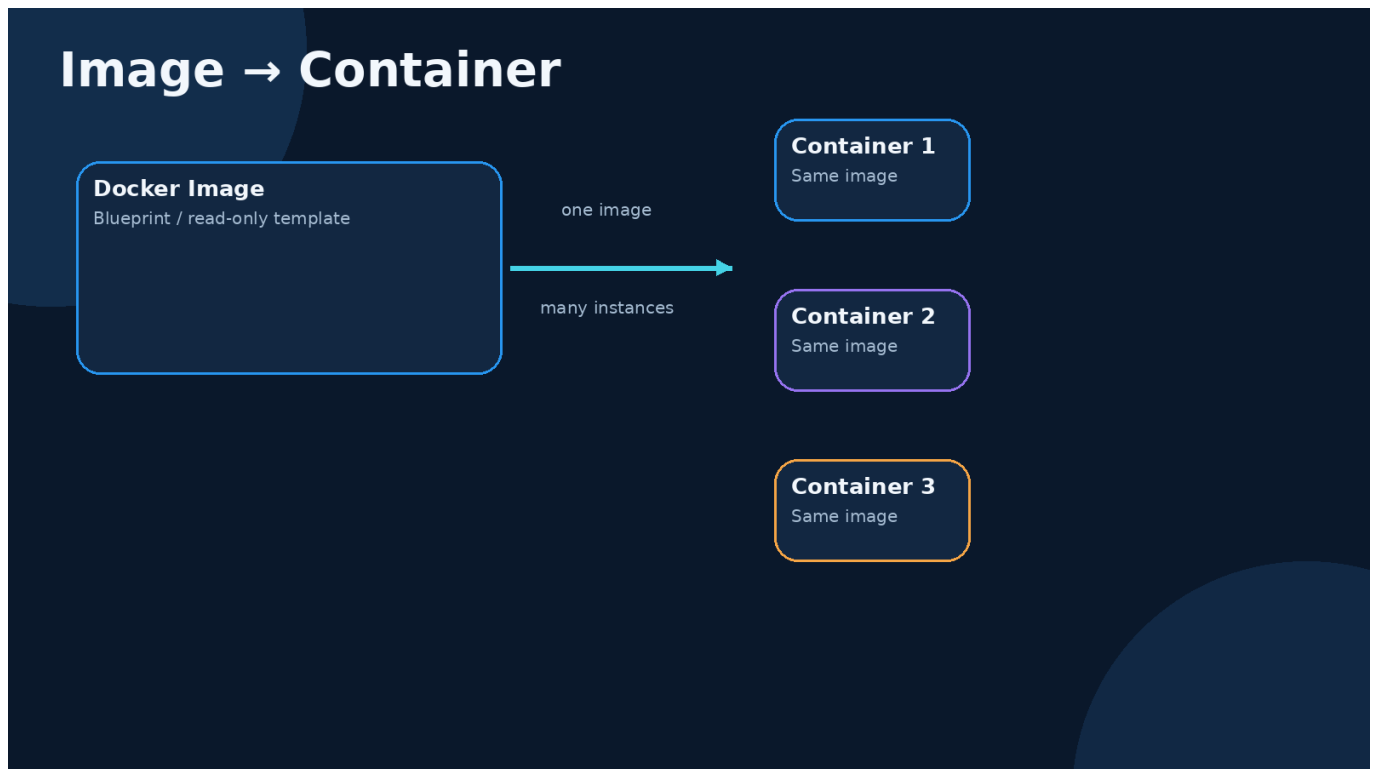
"It works on my machine!"

Docker
Package the app + dependencies into isolated, portable containers.

Key idea: package an application together with its dependencies so the environment can be reproduced more consistently.

2. What is Docker? What is a Container?

The video explains a container as a single bundle containing an application and its dependencies. This makes it easier to share and reproduce the same environment on another machine.



Container

- A running, isolated instance created from a Docker image.
- Packages application code and required dependencies into one unit.
- Lightweight enough to run many isolated applications on one host.
- Different containers can use different runtime/application versions without directly conflicting.

Image vs Container

Image	Container
Blueprint / executable package	Running instance of an image
Static/read-only layers	Running instance with a writable container layer
Can create many containers	Consumes runtime resources
Example: node:20	Example: running Node.js app

3. Docker vs Virtual Machines

The lesson compares containers with virtual machines and emphasizes the lighter overhead of containers. Containers can share the host OS kernel, while VMs include a guest operating system.

Docker Containers vs Virtual Machines

Virtual Machines

Each VM includes a guest operating system.

VM 1
App + Guest OS

VM 2
App + Guest OS

VM 3
App + Guest OS

Docker

Containers share the host OS kernel.

Container 1
App + dependencies

Container 2
App + dependencies

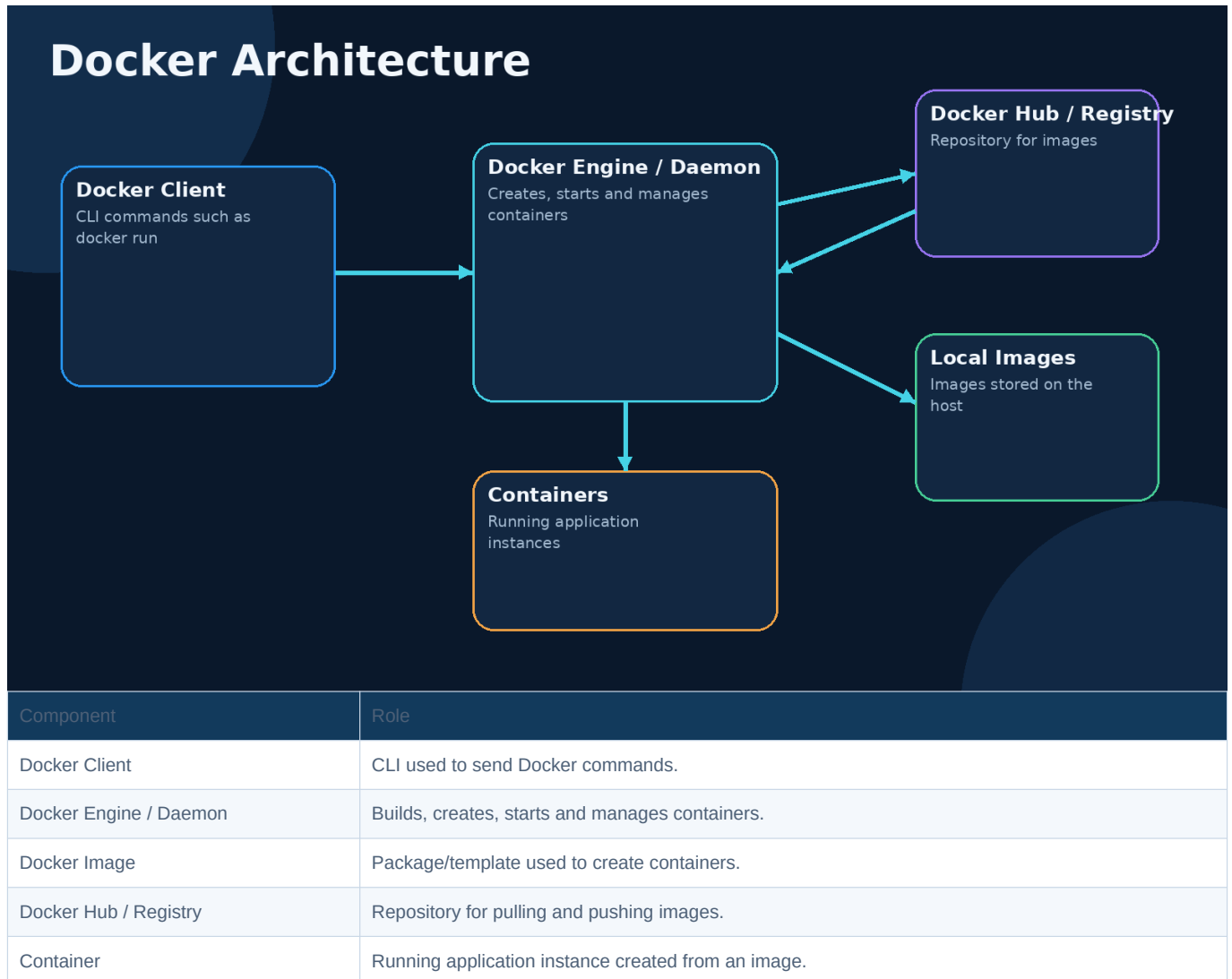
Container 3
App + dependencies

Aspect	Containers	Virtual Machines
Isolation	Application-level	Full machine / OS
Guest OS	Not per container	Each VM has one
Overhead	Generally lower	Generally higher
Startup	Fast	Usually slower
Use	Many lightweight services	Full OS environments

Memory tip: VM = a whole computer in a box. Container = an isolated application environment.

4. Docker Architecture

Docker uses a client/daemon model. The client sends commands to the Docker daemon/engine, which manages images and containers and can obtain images from registries such as Docker Hub.



5. Installation & First Commands

The video demonstrates Docker Desktop installation and then verifies Docker from the command line.

Basic workflow

```
docker pull hello-world
docker images
docker run hello-world
docker ps
docker ps -a
```

- **docker pull** downloads an image from a registry when needed.
- **docker images** lists local images.
- **docker run** creates and starts a container.
- **docker ps** lists running containers.
- **docker ps -a** lists all containers, including stopped ones.

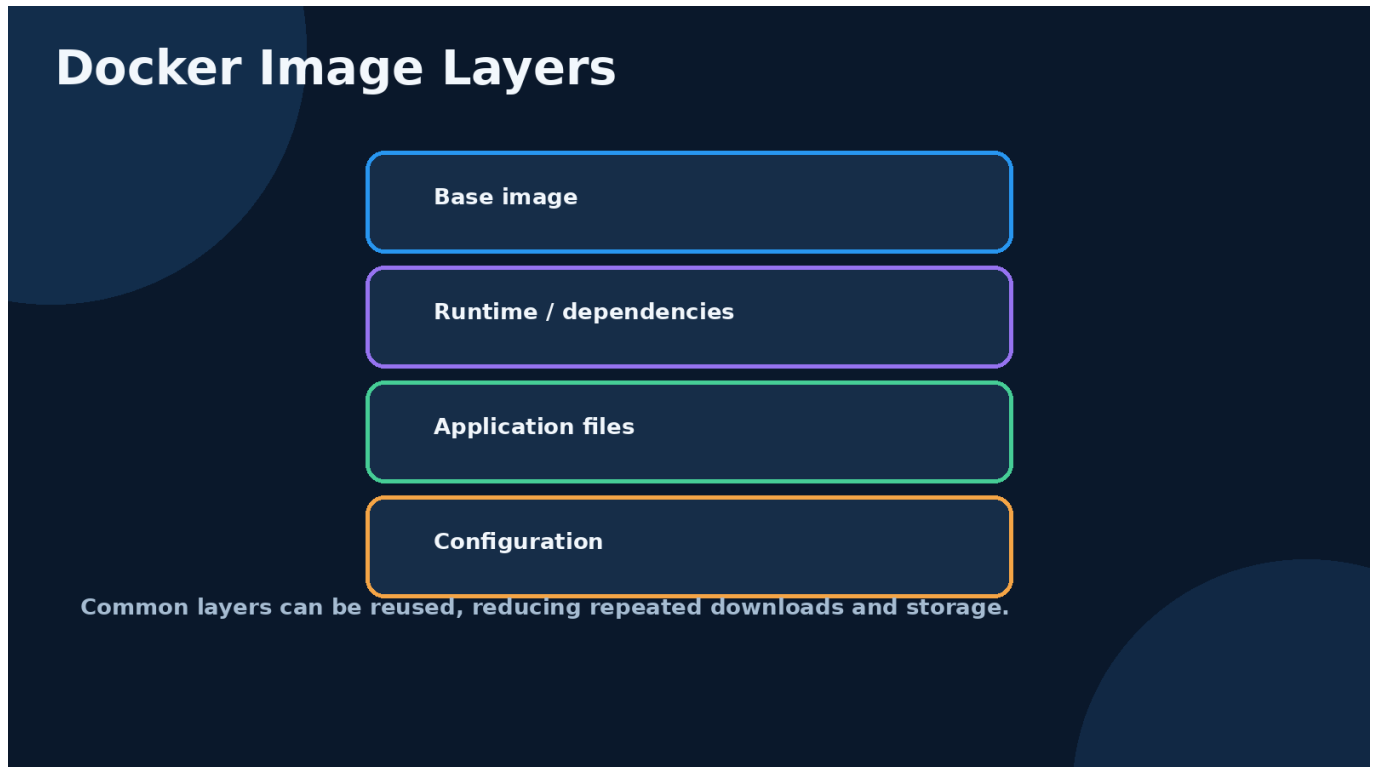
Cleanup

```
docker rm <container_name_or_id>
docker rmi <image name>
```

If an image is still referenced by a container, remove the relevant container first.

6. Image Tags, Versions & Layers

Tags identify image versions or variants. The lesson uses database image versions to show that multiple tags can coexist and that Docker images are composed of reusable layers.



- Example: **mysql:8.0** identifies the MySQL 8.0 tag.
- Different tags can represent different versions or variants.
- Images are composed of layers created from build instructions.
- Existing common layers can be reused instead of downloaded again.
- One image can create multiple containers.

Why layers matter

Layering helps Docker optimize storage and image transfer and can make rebuilding more efficient when earlier layers are unchanged.

7. Running Containers: Ports, Environment Variables & Names

The lesson demonstrates options that make database containers reachable from the host and configurable at startup.

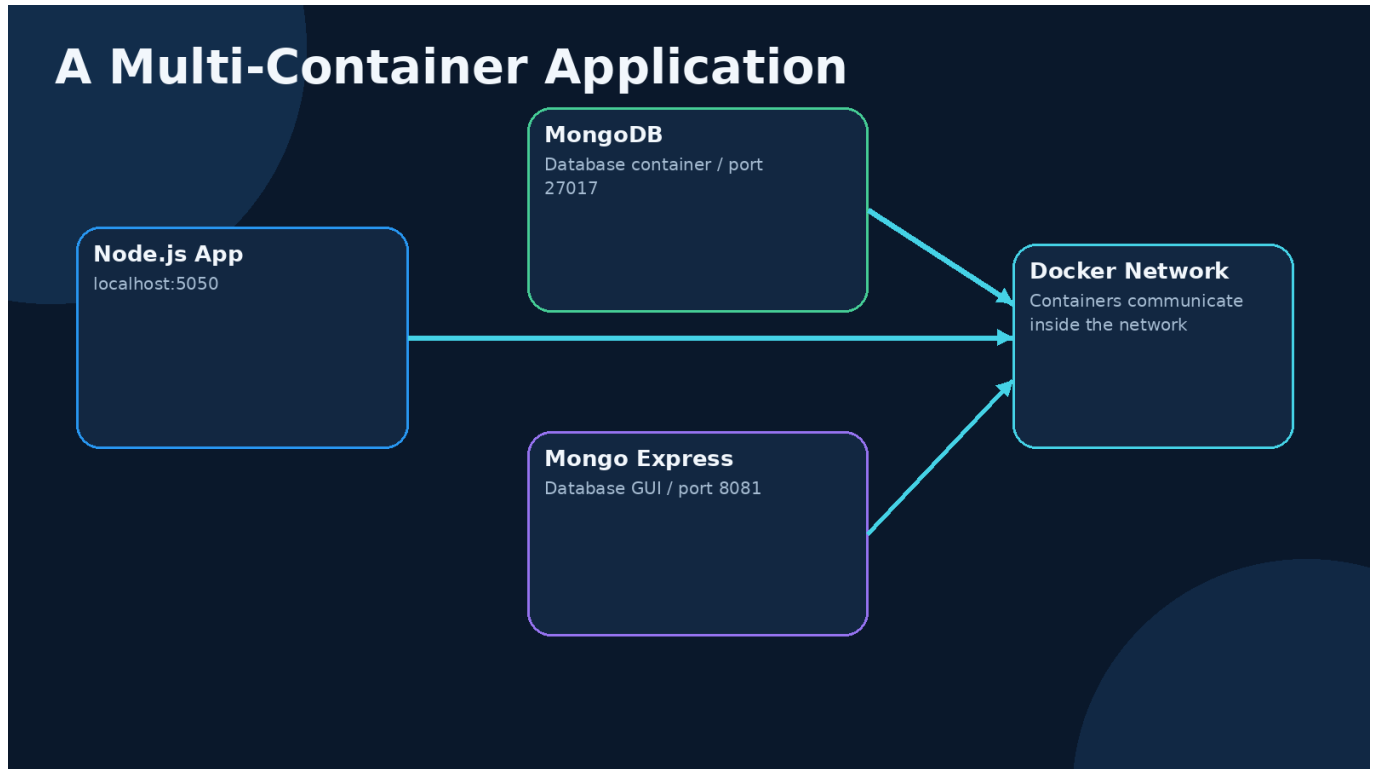
```
docker run -d <image>
docker run -d --name <container_name> <image>
docker run -d -p <host_port>:<container_port> <image>
docker run -d -e <NAME>=<VALUE> <image>
```

Option	Meaning
-d	Detached/background mode.
--name	Give a readable custom container name.
-p host:container	Map a host port to a container port.
-e NAME=value	Pass an environment variable into the container.

In the MongoDB setup, port 27017 is mapped and credentials are passed through environment variables.

8. Docker Networking & Multi-Container Apps

The lesson builds a small multi-container setup using MongoDB and Mongo Express. Docker networking lets the containers communicate inside an isolated network.

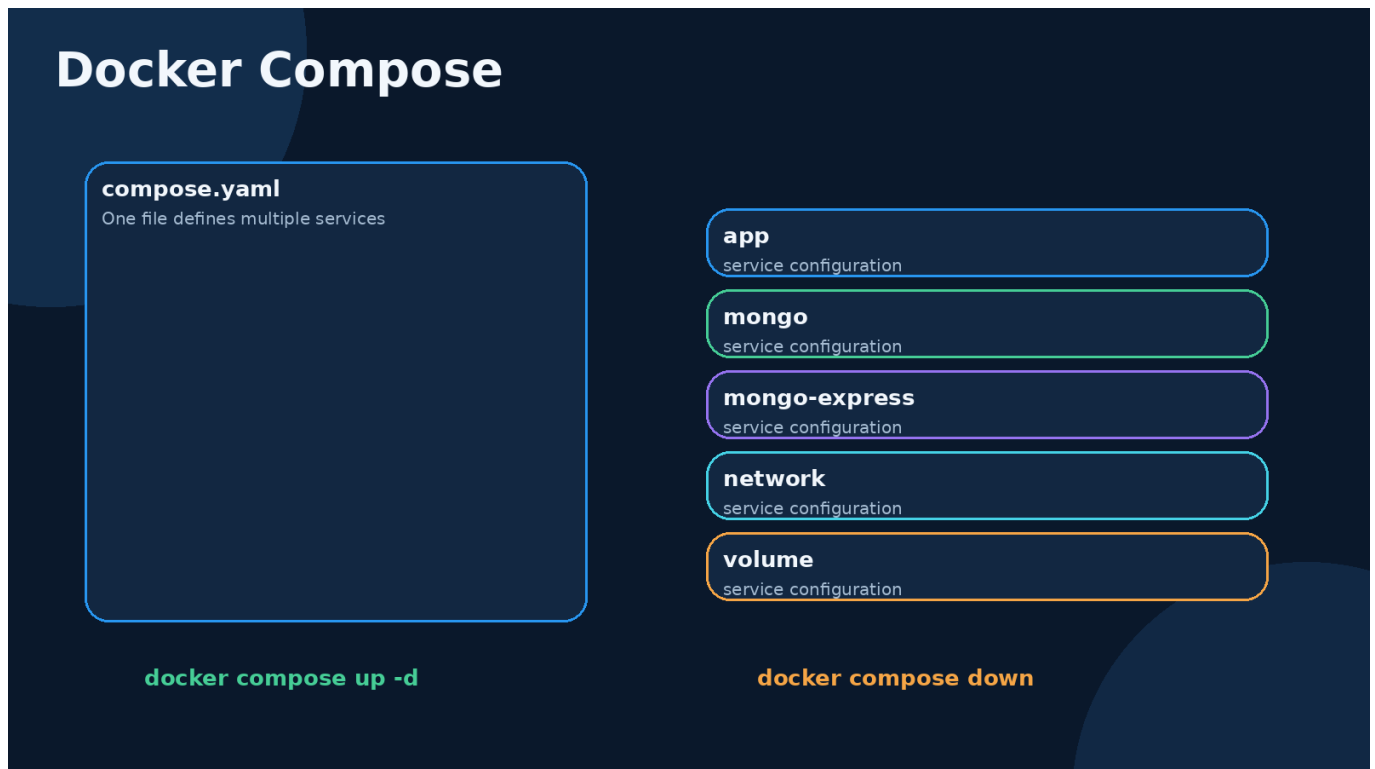


```
docker network ls
docker network create mongo_network
docker run --network mongo_network ...
```

- Containers attached to the same Docker network can communicate with each other.
- Networking separates application components while allowing controlled communication.
- MongoDB acts as the database and Mongo Express provides a browser-based GUI.

9. Docker Compose — Manage Multiple Containers

Docker Compose lets you define multiple services in a YAML file instead of repeatedly typing long terminal commands.



Typical structure

```
services:
  app:
    image: <app-image>
    ports:
      - "5050:5050"

  mongo:
    image: mongo
    ports:
      - "27017:27017"

  mongo-express:
    image: mongo-express
    ports:
      - "8081:8081"
```

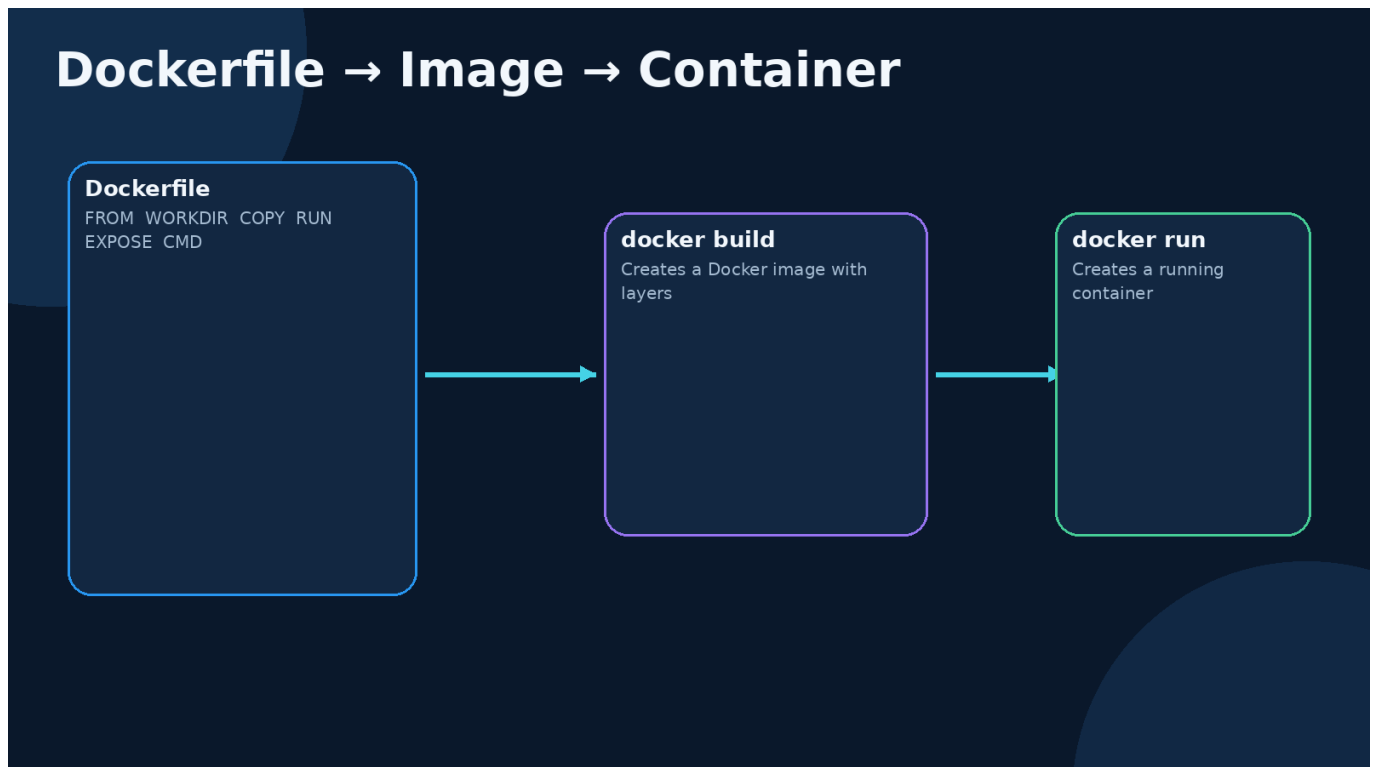
Core commands

```
docker compose up -d
docker compose down
```

- **up** creates/starts the services.
- **down** stops and removes Compose-managed containers and network.
- Compose creates a project network for the services.
- YAML indentation is important.

10. Dockerfile — Build Your Own Image

A Dockerfile is the blueprint used to dockerize an application. Its instructions are used to build the image in layers.



Main instructions

Instruction	Purpose
FROM	Choose the base image.
WORKDIR	Set the working directory inside the image/container.
COPY	Copy host build-context files into the image.
RUN	Execute commands during image build.
EXPOSE	Document the port the application listens on.
ENV	Set environment variables in the image.
CMD	Default command executed when a container starts.

Example

```
FROM node:<version>
WORKDIR /app
COPY . .
RUN npm install
EXPOSE 3000
CMD ["node", "server.js"]
```

11. Build, Run, Inspect & Enter

After creating a Dockerfile, the lesson builds a custom image and then creates a container from it.

```
docker build -t test-app .
docker images
docker run test-app
docker run -it test-app bash
```

Command	Purpose
docker build -t name .	Build an image from the Dockerfile in the current directory.
docker images	Verify the image exists locally.
docker run name	Create and start a container.
docker run -it name bash	Run interactively and open a shell if supported.
docker logs	View container output/logs.
docker inspect	Inspect container configuration/details.

The final . means the current directory is the Docker build context.

12. Docker Hub — Share Your Image

The lesson demonstrates creating a Docker Hub repository, logging in, pushing an image and pulling it back.

```
docker login
docker logout
docker push <username>/<image_name>
docker pull <username>/<image_name>
```

- Create a repository on Docker Hub.
- Tag/build the local image so its name matches the repository path.
- Log in from the terminal.
- Push the image; Docker uploads its layers.
- Another machine can pull the image and run a container from it.

Deployment idea

Docker turns the application environment into a shareable artifact, reducing the need to manually reinstall the same dependencies on another machine.

13. Volumes & Data Persistence

The lesson demonstrates that container data can disappear when the container is removed, while data stored in an attached volume can persist.



Volume types

Type	Description
Named volume	Docker-managed volume with a specific name; useful for persistent data.
Anonymous volume	Volume without a user-defined name; useful for temporary storage.
Bind mount	Maps a host directory/path directly into a container.

Commands

```
docker volume ls
docker volume create my_volume
docker volume prune
docker run --volume my_volume:/data <image>
docker run --mount type=volume,src=my_volume,dst=/data <image>
```

14. Complete Docker Workflow — Remember This

Use this mental model for revision:

Step	Flow
1	Write application code
2	Create a Dockerfile
3	docker build → create an image
4	docker run → create a container
5	Add ports / environment variables / networking as needed
6	Use Compose for multiple containers
7	Use volumes when data must persist
8	Push the image to Docker Hub for sharing/deployment

Viva / interview questions

- **What is Docker?** A containerization platform for packaging and running applications with dependencies.
- **Image vs container?** Image is the blueprint/package; container is the running instance.
- **Why Docker?** Consistency, portability, isolation and easier deployment.
- **Why Compose?** To define and run multi-container applications together.
- **Why volumes?** To keep data outside the temporary container lifecycle.
- **What is Dockerfile?** Instructions used to build a Docker image.
- **What does -p do?** Maps a host port to a container port.
- **What does -d do?** Runs a container in detached mode.
- **What are image layers?** Reusable parts that together form an image.

15. Quick Command Cheat Sheet

Task	Command
List images	docker images
Pull image	docker pull
Run container	docker run
Run detached	docker run -d
List running containers	docker ps
List all containers	docker ps -a
Stop container	docker stop
Remove container	docker rm
Remove image	docker rmi
View logs	docker logs
Inspect container	docker inspect
List networks	docker network ls
Create network	docker network create
List volumes	docker volume ls
Create volume	docker volume create
Build image	docker build -t .
Compose start	docker compose up -d
Compose stop/remove	docker compose down
Push image	docker push /
Login	docker login

Best study sequence: **Image** → **Container** → **ports + environment variables** → **networking + Compose** → **Dockerfile + build** → **volumes + Docker Hub**.